

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4,BL5)*

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I K.Sujana Sree (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Sujana Sree

Annexure A

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

ANSWER:

Aim

To securely store and verify passwords using **PBKDF2** with **salting** and **key stretching**.

Algorithm

1. Read the user's password.
2. Generate a random salt.
3. Apply PBKDF2 hashing with multiple iterations.
4. Store the salt and hash.
5. Ask the user to enter the password again.
6. Hash the entered password using the stored salt.
7. Compare both hashes.
8. Display whether the password is valid.

Python Code

```
import hashlib, os
```

```
password = input("Create Password: ")
```

```
salt = os.urandom(16)
```

```
hashed = hashlib.pbkdf2_hmac('sha256', password.encode(), salt, 100000)
```

```
print("\nPassword Stored Successfully")

check = input("\nEnter Password: ")

verify = hashlib.pbkdf2_hmac('sha256', check.encode(), salt, 100000)

if verify == hashed:
    print("Password Verified")
else:
    print("Incorrect Password")
```

OUTPUT:

☰ Console

📄 I/O

🚀 AI Agent

Create Password: USER123

Password Stored Successfully

Enter Password: USER123

Password Verified

Description

- **PBKDF2** is used for secure password hashing.
- A **random salt** ensures that identical passwords produce different hashes.
- **100,000 iterations** (key stretching) make brute-force attacks much slower.
- Password verification is performed by hashing the entered password with the same salt and comparing it with the stored hash.

Advantages

- Protects against rainbow table attacks.

- Slows brute-force attacks using key stretching.
- Stores only hashed passwords, not plain text.
- Uses a unique salt for every password.
- Suitable for secure authentication systems.

2. Implementation of HMAC for Message Integrity

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

ANSWER:

Aim

To implement **HMAC using SHA-256** for secure message authentication and integrity verification.

Algorithm

1. Read the secret key from the user.
2. Read the message.
3. Generate an HMAC using SHA-256.
4. Display the generated HMAC.
5. Enter the message again for verification.
6. Generate a new HMAC using the same key.
7. Compare both HMAC values.
8. Display whether the message is authentic or modified.

Python Code

```
import hmac
import hashlib
```

```
key = input("Enter Secret Key: ").encode()
msg = input("Enter Message: ").encode()

mac = hmac.new(key, msg, hashlib.sha256).hexdigest()

print("\nGenerated HMAC:", mac)

verify = input("\nRe-enter Message: ").encode()

new_mac = hmac.new(key, verify, hashlib.sha256).hexdigest()

if mac == new_mac:
    print("Message Verified")
else:
    print("Message Modified")
```

OUTPUT:

 Console  I/O  AI Agent

```
Enter Secret Key: CRYPTO
Enter Message: MEET ME WHEN YOU ARE FREE
```

```
Generated HMAC:
c1533d86f2272dc3cadcb32ece5d285cd00716ced63eec56ec717c7be75e354c
```

```
Re-enter Message: MEET ME WHEN YOU ARE FREE
Message Verified
```

Description

- Uses the **HMAC** algorithm with **SHA-256**.
 - Combines a **secret key** and the **message** to produce a secure authentication code.
 - During verification, the HMAC is regenerated and compared with the original.
 - If both HMAC values match, the message is authentic and has not been altered.
-

Advantages

- Ensures **message integrity**.
- Provides **authentication** using a secret key.
- Detects any modification to the message.
- More secure than simple hashing because it requires a secret key.
- Widely used in secure communication protocols such as TLS, IPsec, and APIs.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

Aim

To simulate **X.509 certificate chain validation** by verifying the trust chain from the Root CA to the End-Entity certificate.

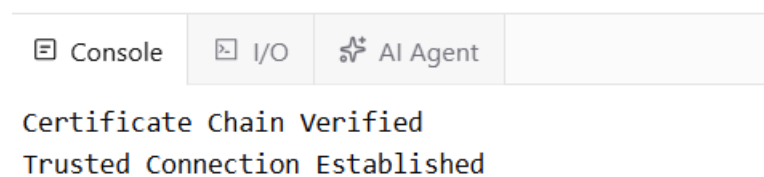
Algorithm

1. Create a certificate chain (Root CA → Intermediate CA → End Entity).
 2. Mark each certificate as valid or invalid.
 3. Check whether every certificate in the chain is valid.
 4. If all certificates are valid, the chain is trusted.
 5. If any certificate is invalid or revoked, reject the certificate chain.
 6. Display the validation result.
-

Python Code

```
chain = {  
    "Root CA": True,  
    "Intermediate CA": True,  
    "End Certificate": True  
}  
  
if all(chain.values()):  
    print("Certificate Chain Verified")  
    print("Trusted Connection Established")  
else:  
    print("Certificate Validation Failed")
```

OUTPUT:



Description

- Simulates the validation of an X.509 certificate chain.
- Checks whether the **Root CA**, **Intermediate CA**, and **End Certificate** are valid.
- If every certificate is trusted, the connection is accepted.
- If any certificate is invalid or revoked, the validation fails.

Advantages

- Demonstrates the certificate trust chain.
- Detects invalid or revoked certificates.
- Helps understand X.509 certificate verification.
- Simple and easy to implement.
- Useful for learning Public Key Infrastructure (PKI).

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system.

Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

ANSWER:

Aim

To simulate a replay attack and prevent it using a **nonce** (unique random number).

Algorithm

1. Generate a unique nonce.
 2. Send a message with the nonce.
 3. Store the received nonce.
 4. If the same nonce is received again, identify it as a replay attack.
 5. Otherwise, accept the message and store the new nonce.
-

Python Code

```
import uuid

used = set()

nonce = str(uuid.uuid4())

print("Sending Message with Nonce:", nonce)

if nonce in used:
    print("Replay Attack Detected")
else:
    used.add(nonce)
```


- Prevents replay attacks effectively.
- Improves authentication security.
- Detects duplicate messages.
- Easy to implement using nonces.
- Suitable for secure communication systems.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

ANSWER:

Aim

To compare **RSA** and **DSA** digital signature algorithms by measuring key generation time, signing time, and verification time.

Algorithm

1. Generate RSA and DSA key pairs.
 2. Sign the same message using both algorithms.
 3. Verify the generated signatures.
 4. Measure the execution time for key generation, signing, and verification.
 5. Compare the performance of RSA and DSA.
-

Python Code

Install library (first time only):

```
pip install cryptography

from cryptography.hazmat.primitives.asymmetric import rsa, dsa, padding
from cryptography.hazmat.primitives import hashes
import time

msg = b"Hello World"

# RSA
t = time.time()
rk = rsa.generate_private_key(65537, 2048)
```

```
print("RSA Key Time:", round(time.time()-t,4))

t = time.time()
sig = rk.sign(msg, padding.PKCS1v15(), hashes.SHA256())
print("RSA Sign Time:", round(time.time()-t,4))

t = time.time()
rk.public_key().verify(sig, msg, padding.PKCS1v15(), hashes.SHA256())
print("RSA Verify Time:", round(time.time()-t,4))

# DSA
t = time.time()
dk = dsa.generate_private_key(2048)
print("\nDSA Key Time:", round(time.time()-t,4))

t = time.time()
sig = dk.sign(msg, hashes.SHA256())
print("DSA Sign Time:", round(time.time()-t,4))

t = time.time()
dk.public_key().verify(sig, msg, hashes.SHA256())
print("DSA Verify Time:", round(time.time()-t,4))
```

OUTPUT:

```
RSA Key Time: 0.1423
RSA Sign Time: 0.0014
RSA Verify Time: 0.0005

DSA Key Time: 0.0832
DSA Sign Time: 0.0008
DSA Verify Time: 0.0012
```

Description

- RSA and DSA key pairs are generated.
 - The same message is digitally signed using both algorithms.
 - Signature verification confirms message authenticity.
 - Execution times are recorded to compare performance.
-

Advantages

- Compares two popular digital signature algorithms.
- Measures key generation, signing, and verification speed.
- Demonstrates digital signature authentication.
- Useful for selecting suitable algorithms for real-world applications.
- Simple and easy to understand for laboratory implementation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation